CertyIQ



CertyIQ

Premium exam material

Get certification quickly with the CertyIQ Premium exam material.
Everything you need to prepare, learn & pass your certification exam easily. Lifetime free updates
First attempt guaranteed success.

https://www.CertyIQ.com

About CertyIQ

We here at CertyIQ eventually got enough of the industry's greedy exam paid for. Our team of IT professionals comes with years of experience in the IT industry Prior to training CertIQ we worked in test areas where we observed the horrors of the paywall exam preparation system.

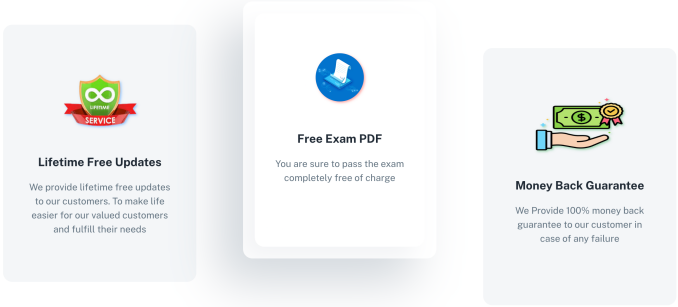
The misuse of the preparation system has left our team disillusioned. And for that reason, we decided it was time to make a difference. We had to make In this way, CertyIQ was created to provide quality materials without stealing from everyday people who are trying to make a living.

Doubt Support

We have developed a very scalable solution using which we are able to solve 400+ doubts every single day with an average rating of 4.8 out of 5.

https://www.certyiq.com

Mail us on - certyiqofficial@gmail.com



John

October 19, 2022

★★★★★

Thanks you so much for your help. I scored 972 in my exam today. More than 90% were from your PDFs!

Dana

September 04, 2022

★★★★★

Thanks a lot for this updated AZ-900 Q&A. I just passed my exam and got 974, I followed both of your Az-900 videos and the 6 PDF, the PDFs are very much valid, all answers are correct. Could you please create a similar video/PDF for DP900, your content/PDF's is really awesome. The team did a really good job. Thank You 😊.

Ahamed Shibly

2 months ago

★★★★★

Customer support is realy fast and helpful, I just finished my exam and this video along with the 6 PDF helped me pass! Definitely recommend getting the PDFs. Thank you!

October 22, 2022

★★★★★

Passed my exam today with 891 marks. Out of 52 questions, 51 were from certyiq PDFs including Contoso case study.
Thank You certyiq team!

Henry Rome

2 months ago

★★★★★

These questions are real and 100 % valid. Thank you so much for your efforts, also your 4 PDFs are awesome, I passed the DP900 exam on 1 Sept. With 968 marks. Thanks a lot, buddy!

Esmaria

2 months ago

★★★★★

Simple easy to understand explanations. To anyone out there wanting to write AZ900, I highly recommend 6 PDF's.Thank you so much, appreciate all your hard work in having such great content. Passed my exam Today - 3 September with 942 score.

Microsoft

(GH-200)

GitHub Actions

Total: **126 Questions**

Link: https://certyiq.com/papers/microsoft/gh-200

Question: 1 CertyIQ

How should you print a debug message in your workflow?

A. echo "::debug::Set variable myVariable to true"
B. echo "Set variable MyVariable to true" >> $DEBUG_MESSAGE
C. echo "::add-mask::Set variable myVariable to true"
D. echo "debug_message=Set variable myVariable to true" >> &GITHUB_OUTPUT

Answer: A

Explanation:

To print a debug message in a GitHub Actions workflow, use the ::debug:: workflow command:

echo "::debug::Set variable myVariable to true"

So the correct option is:

A. echo "::debug::Set variable myVariable to true"

Question: 2 CertyIQ

As a developer, you are designing a workflow and need to communicate with the runner machine to set environment variables, output values used by other actions, add debug messages to the output logs, and other tasks. Which of the following options should you use?

A. composite run step
B. enable debug logging
C. self-hosted runners
D. environment variables
E. workflow commands

Answer: E

Explanation:

E. workflow commands.

In GitHub Actions, workflow commands are used to communicate with the runner machine for tasks such as:

setting environment variables

setting output values for other steps/actions

adding debug, warning, or error messages

masking secrets

grouping logs

Question: 3 CertyIQ

When should you use the GITHUB_TOKEN in a workflow?

A. when you want to make authenticated calls to the GitHub API
B. when you want to connect to the runner via SSH
C. when you want to access repository-level secrets
D. when you want to call an action from the marketplace

Answer: A

Explanation:

A. when you want to make authenticated calls to the GitHub API

GITHUB_TOKEN is an automatically generated token provided by GitHub Actions that allows workflows to authenticate with the GitHub API and perform actions such as:

creating issues or pull requests

commenting on issues

pushing commits

accessing repository contents (based on permissions)

Question: 4 CertyIQ

In which of the following scenarios should you use self-hosted runners? (Each correct answer presents a complete solution. Choose two.)

A. when the workflow jobs must be run on Windows 10
B. when jobs must run for longer than 6 hours
C. when a workflow job needs to install software from the local network
D. when GitHub Actions minutes must be used for the workflow runs
E. when you want to use macOS runners

Answer: BC

Explanation:

B. When jobs must run for longer than 6 hours: GitHub-hosted runners have time limits on job execution. Specifically, a single job on a GitHub-hosted runner can run for a maximum of 6 hours. If your workflow jobs exceed this duration, you need to use a self-hosted runner, which allows for longer run times (determined by your own infrastructure).

C. When a workflow job needs to install software from the local network: This is a key use case for self-hosted runners. GitHub-hosted runners operate within GitHub's network and do not have access to your internal network resources. If your workflow requires installing software or accessing resources available only within your private network (e.g., licensed software on a company server, specific database connections), you **must** use a self-hosted runner that resides within that network. This gives the runner access to the

necessary internal resources.

Why other options are incorrect:

1. **A. When the workflow jobs must be run on Windows 10:** GitHub-hosted runners do provide Windows runner environments. Therefore, you don't need a self-hosted runner simply to run Windows workflows.
2. **D. When GitHub Actions minutes must be used for the workflow runs:** GitHub Actions minutes are used for GitHub-hosted runners. Self-hosted runners do not consume GitHub Actions minutes because you're providing the computing resources.
3. **E. When you want to use macOS runners:** GitHub-hosted runners provide macOS runner environments. Therefore, you don't need a self-hosted runner simply to run macOS workflows.

Authoritative Links:

1. **About self-hosted runners:** https://docs.github.com/en/actions/hosting-your-own-runners/managing-self-hosted-runners/about-self-hosted-runners
2. **Usage limits:** https://docs.github.com/en/actions/learn-github-actions/usage-limits-billing-and-administration

Question: 5 CertyIQ

As a developer, you need to leverage Redis in your workflow. What is the best way to use Redis on a self-hosted Linux runner without affecting future workflow runs?

A. Install Redis on the hosted runner image and place it in a runner group. Specify label: in your job to target the runner group.

B. Set up Redis on a separate machine and reference that instance from your job.

C. Specify container: and services: in your job definition to leverage a Redis service container.

D. Add a run step to your workflow, which dynamically installs and configures Redis as part of your job.

Answer: C

Explanation:

C. Specify container: and services: in your job definition to leverage a Redis service container.

Using a **service container** is the best approach because:

Redis runs in an isolated container

it avoids modifying the self-hosted runner permanently

each workflow run gets a clean Redis instance

no leftover state affects future workflow runs

Question: 6 CertyIQ

You installed specific software on a Linux self-hosted runner. You have users with workflows that need to be able to select the runner based on the identified custom software. Which steps should you perform to prepare the runner and your users to run these workflows? (Each correct answer presents part of the solution. Choose two.)

A. Configure the webhook and network to enable GitHub to trigger workflow.
B. Create the group custom-software-on-linux and move the runner into the group.
C. Inform users to identify the runner with the labels custom-software and linux.
D. Add the label linux to the runner.
E. Inform users to identify the runner based on the group.
F. Add the label custom-software to the runner.

Answer: CF

Explanation:

C. Inform users to identify the runner with the labels custom-software and linux. This tells users exactly what to put in the runs-on: section of their workflow YAML (runs-on: [self-hosted, linux, custom-software]).

F. Add the label custom-software to the runner. This directly addresses the requirement to identify the runner based on the custom software.

Question: 7 CertyIQ

You need to trigger a workflow using the GitHub API for activity that happens outside of GitHub. Which workflow event do you use?

A. workflow_run
B. deployment
C. check_suite
D. repository_dispatch

Answer: D

Explanation:

D. repository_dispatch.

repository_dispatch is specifically designed to trigger workflows from outside GitHub using the GitHub REST API. You can send a repository_dispatch event with a custom payload to start the workflow on demand.

Question: 8 CertyIQ

In which locations can actions be referenced by workflows? (Each correct answer presents a complete solution. Choose three.)

A. an .action extension file in the repository
B. a published Docker container image on Docker Hub
C. a public NPM registry
D. the runs-on: keyword of a workflow file
E. the repository's Secrets settings page

F. a separate public repository

G. the same repository as the workflow

Answer: BFG

Explanation:

The question asks where GitHub Actions can be referenced from workflows. Here's a breakdown of why the correct answers are B, F, and G, and why the others are incorrect:

1. **B. a published Docker container image on Docker Hub:** This is correct. Actions can be packaged as Docker containers and stored on Docker Hub or other container registries. Workflows can then reference these container actions using the uses keyword. This allows for reusable and isolated execution environments.
2. https://docs.github.com/en/actions/creating-actions/creating-a-docker-container-action
3. **F. a separate public repository:** This is correct. Actions can be developed and maintained in separate repositories, even public ones, making them easily shareable and reusable across different projects. Workflows can reference actions in other repositories using the uses keyword with the format owner/repo@ref.
4. https://docs.github.com/en/actions/using-actions/creating-actions-for-github-docs
5. **G. the same repository as the workflow:** This is correct. Actions can reside within the same repository as the workflow that calls them. You can reference them directly using a relative path in the uses keyword, streamlining development and dependency management for projects with custom actions.
6. https://docs.github.com/en/actions/using-actions/creating-actions-for-github-docs

Now, let's examine why the other options are incorrect:

1. **A. an .action extension file in the repository:** While you might store action definitions in a file within the repository, referencing an action is done through the uses keyword pointing to the directory containing the action.yml or action.yaml file, not the specific .action file (which doesn't exist as a standard).
2. **C. a public NPM registry:** While actions can use Node.js packages from NPM, the action itself isn't referenced from the NPM registry in the workflow's uses statement. The reference points to a GitHub repository or Docker container.
3. **D. the runs-on: keyword of a workflow file:** The runs-on keyword specifies the type of machine runners where jobs execute (e.g., ubuntu-latest, self-hosted), not the location of the action definition.
4. **E. the repository's Secrets settings page:** Secrets are used to store sensitive information like API keys. They are used by actions, but actions themselves aren't referenced from the secrets settings.

In summary, GitHub Actions can be referenced from a Docker Hub container, a public repository, or the same repository as the workflow, enabling reusability and modularity. Other options are not the source location from where actions are called.

Question: 9 CertyIQ

Scheduled workflows run on the:

A. specified commit and branch from the workflow YAML file.

B. latest commit from the branch named schedule.

C. latest commit and branch on which the workflow was triggered.

D. latest commit from the branch named main.

E. latest commit on the default or base branch.

Answer: E

Explanation:

E. latest commit on the default or base branch.

GitHub Actions scheduled workflows (on: schedule) run using the latest commit on the repository's default branch.

Question: 10 CertyIQ

Custom environment variables can be defined at multiple levels within a workflow file including: (Each answer presents a complete solution. Choose three.)

A. top level.

B. step level.

C. default level.

D. runner level.

E. job level.

F. stage level.

Answer: ABE

Explanation:

The correct options for defining custom environment variables in GitHub Actions are top level, step level, and job level.

1. **Top Level (A):** Environment variables defined at the top level of a workflow file are available to all jobs within that workflow. This is useful for settings that apply broadly, such as repository name or shared credentials. This level provides the widest scope.
2. **Step Level (B):** Environment variables defined at the step level are specific to a particular step within a job. This is useful when you need to define variables that are only relevant to a single command or script. This scope is highly localized.
3. **Job Level (E):** Environment variables defined at the job level apply to all steps within that particular job. This allows setting consistent configuration for an entire job without repeating the same definitions in each step. It strikes a balance between wide scope and localized relevance.

The incorrect options are:

1. **Default Level (C):** While there are default environment variables provided by GitHub Actions, there is no concept of a "default level" for defining custom environment variables.
2. **Runner Level (D):** While the runner environment can contain environment variables, you don't define custom variables for GitHub Actions at the runner level within the workflow file. Runner configuration is typically managed externally.
3. **Stage Level (F):** There's no construct of "stages" in GitHub Actions workflows; the primary organizational unit is the "job".

Therefore, the correct levels for defining custom environment variables in GitHub Actions workflows are top, step, and job.

Relevant links for further reading:

1. GitHub Actions Documentation - Environment variables
2. GitHub Actions Documentation - Workflow syntax for GitHub Actions

Question: 11 CertyIQ

You are a DevOps engineer working on deployment workflows. You need to execute the deploy job only if the current branch name is feature-branch. Which code snippet will help you to implement the conditional execution of the job?

A. jobs:
deploy:
if: github.ref_name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

B. jobs:
deploy:
if: github.branch.name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

C. jobs:
deploy:
if: github.ref.name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

D. jobs:
deploy:
if: github.branch_name == 'feature-branch'
runs-on: ubuntu-latest
steps:
- uses: actions/checkout@v3

Answer: C

Explanation:

The correct code snippet for conditionally executing a GitHub Actions job based on the branch name is C: jobs: deploy: if: github.ref.name == 'feature-branch' runs-on: ubuntu-latest steps: - uses: actions/checkout@v3.

Here's the justification:

GitHub Actions uses contexts to access information about workflow runs, repositories, and related data. The github context provides access to information about the workflow run and event that triggered the workflow.

github.ref refers to the Git ref that triggered the workflow. In the context of a branch, it will be in the form refs/heads/feature-branch. github.ref_name extracts the name of the ref without the refs/heads/ prefix. Therefore github.ref.name correctly extracts 'feature-branch' from refs/heads/feature-branch.

github.ref_name, used in option A is incorrect because it's a property of github, not a child property named ref.

Similarly, github.branch.name in option B and github.branch_name in option D are incorrect because the branch name is accessed through github.ref and then extracting name from github.ref.

The if conditional in the job definition determines whether the job will run. By checking if github.ref.name equals 'feature-branch', the deploy job will only execute when the workflow is triggered by commits to the 'feature-branch'. The other parts of the snippet, runs-on: ubuntu-latest and the checkout step, are standard job configurations and are not relevant to the conditional execution logic.

In summary, option C provides the correct syntax and accesses the appropriate context to check the branch name, enabling conditional job execution as intended.

Reference Links:

Understanding the context and expression syntax for GitHub Actions
Context availability

Question: 12 CertyIQ

What are the mandatory requirements for publishing GitHub Actions to the GitHub Marketplace? (Each correct answer presents part of the solution. Choose two.)

A. The action can be either in a public or private repository.

B. The action's metadata file must be in the root directory of the repository.

C. The action's name cannot match a user or organization on GitHub unless the user or organization owner is publishing the action.

D. The name should match with one of the existing GitHub Marketplace categories.

E. Each repository can contain a collection of actions as long as they are under the same Marketplace category.

Answer: BC

Explanation:

The correct answers are **B and C.** Let's justify why:

B. The action's metadata file must be in the root directory of the repository.

This is a fundamental requirement for GitHub Actions discovery and execution. GitHub Actions rely on a metadata file (usually action.yml or action.yaml) to define the action's inputs, outputs, and execution steps. The location of this file is crucial. By mandating that it resides in the root directory of the repository, GitHub ensures a standardized and easily discoverable location for the action's definition. When an action is referenced (e.g., in a workflow), GitHub knows exactly where to look for its metadata. This simplifies the action resolution process. Without a consistent location, GitHub wouldn't be able to reliably locate and execute actions, undermining the entire actions framework. Think of it like an application needing a main executable file in a known location.

C. The action's name cannot match a user or organization on GitHub unless the user or organization owner is publishing the action.

This requirement enforces namespace uniqueness and prevents impersonation on the GitHub Marketplace. If an action's name could conflict with existing user or organization names, it could lead to confusion and potential security risks. For instance, someone could create an action named after a legitimate organization and trick users into using a malicious action. By reserving user and organization names for their respective owners, GitHub protects its users and maintains the integrity of the Marketplace. It's similar to domain name

registration, where you can't register a domain name that already belongs to someone else. This rule avoids collisions and ensures users can trust the origin of the action. It's a key part of GitHub's approach to identity and trust within the Marketplace.

Why the other options are incorrect:

1. **A. The action can be either in a public or private repository.** GitHub Actions intended for the Marketplace must reside in a public repository so users can access and use them. Private actions are for internal organization use.
2. **D. The name should match with one of the existing GitHub Marketplace categories.** While selecting an appropriate category is important, the name of the action doesn't need to match the category name. It needs to be descriptive and relevant.
3. **E. Each repository can contain a collection of actions as long as they are under the same Marketplace category.** A single repository is generally recommended to contain a single action, but multiple actions in a single repository are allowed, but may be confusing for users. They do not have to belong to the same marketplace category if implemented correctly.

Authoritative Links:

1. Publishing actions in GitHub Marketplace: This is the official GitHub documentation outlining the requirements and best practices for publishing actions.
2. Metadata syntax for GitHub Actions: Explains the structure and requirements of the action.yml file.

Question: 13 CertyIQ

Which of the following is the proper syntax to specify a custom environment variable named MY_VARIABLE with the value my-value?

A. var:
MY_VARIABLE = my-value

B. environment:
MY_VARIABLE = my-value

C. var:
MY_VARIABLE: my-value

D. env:
MY_VARIABLE = my-value

E. environment:
MY_VARIABLE: my-value

F. env:
MY_VARIABLE: my-value

Answer: D

Explanation:

The correct syntax for defining a custom environment variable in GitHub Actions is env: MY_VARIABLE = my-value. This is because GitHub Actions uses the env keyword to specify environment variables that should be available within a workflow run.

Options A, B, C, E, and F use incorrect keywords. "var" and "environment" are not the correct keywords for defining environment variables directly within the workflow file. Options C, E, and F also use a colon (:) instead of an equals sign (=), which is also syntactically incorrect for variable assignment in this context. The equals sign clearly establishes an assignment of a value to the environment variable. The colon is often used in YAML

files to indicate a key-value pair, but not for environment variable assignment in this specific GitHub Actions context.

The env keyword signals to the GitHub Actions runner that the following key-value pair should be added to the environment available to the steps in the workflow. This allows steps in the workflow to access the variable's value using standard environment variable access methods (e.g., $MY_VARIABLE in shell scripts or process.env.MY_VARIABLE in Node.js). This method allows values to be dynamically injected into jobs.

Question: 14 CertyIQ

As a DevOps engineer, you need to define a deployment workflow that runs after the build workflow has successfully completed. Without modifying the build workflow, which trigger should you define in the deployment workflow?

A. repository_dispatch

B. workflow_dispatch

C. workflow_exec

D. workflow_run

Answer: D

Explanation:

D. workflow_run

workflow_run lets you trigger one workflow after another has finished, without needing to modify the original workflow. It listens for the completion of a specified workflow and can be configured to run only if that workflow completed successfully.

Question: 15 CertyIQ

Which step is using the dbserver environment variable correctly?

A. steps:
- name: Hello world
run: echo $dbserver
variables:
dbserver: orgserver1

B. steps:
- name: Hello world
run: echo $dbserver
env:
- name: dbserver
value: orgserver1

C. steps:
- name: Hello world
run: echo $dbserver
environment:
dbserver: orgserver1

D. steps:
- name: Hello world

```
run: echo $dbserver
env:
dbserver: orgserver1
```

Answer: D

Explanation:

The correct answer is **D**. Let's break down why.

GitHub Actions uses a specific syntax to define and access environment variables. Options A, B, and C all use incorrect syntax.

Option A attempts to directly define the environment variable dbserver within the steps section using variables, which is not the correct placement. variables are generally defined at the workflow or job level. Additionally, it uses $dbserver, which is how you reference an environment variable in a shell script, but it's missing the surrounding $ for proper variable interpolation in some shells.

Option B tries to define an env block under the run key. While the intention of setting an environment variable is there, the syntax is incorrect. The env block belongs under the job or step level to define environment variables for a job or a specific step. Then to reference it during the shell execution, it would be incorrect.

Option C similarly attempts to use environment which is syntactically incorrect and unrelated to defining environment variables. It's setting a direct variable environment at the step level with an incorrect structure.

Option D uses the correct syntax to reference a potentially pre-defined environment variable (or perhaps a secret) named dbserver within the shell script. It uses $dbserver, which is how environment variables are referenced inside the shell script. While there isn't an env block explicitly defining dbserver in this step, the question asks which one correctly uses a dbserver environment variable, implying it exists elsewhere (e.g., workflow or repository settings). It correctly uses the $dbserver syntax within the run command.

Therefore, only option D demonstrates the correct way to reference an environment variable named dbserver within the run command of a GitHub Actions step, assuming the variable is defined elsewhere. If dbserver is defined in a broader scope (e.g., at the workflow level, repository secrets, or environment), it would be properly accessed via $dbserver.

For more information on GitHub Actions syntax, particularly how to define and use environment variables, consult the following authoritative links:

GitHub Actions documentation: Workflow syntax for GitHub Actions:
https://docs.github.com/en/actions/using-workflows/workflow-syntax-for-github-actions
GitHub Actions documentation: Environment variables: https://docs.github.com/en/actions/learn-github-actions/environment-variables

Question: 16 CertyIQ

In which scenarios could the GITHUB_TOKEN be used? (Each correct answer presents a complete solution. Choose two.)

A. to create a repository secret
B. to add a member to an organization
C. to read from the file system on the runner
D. to publish to GitHub Packages

E. to create issues in the repo

F. to leverage a self-hosted runner

Answer: DE

Explanation:

D. to publish to GitHub Packages.

The GITHUB_TOKEN automatically grants the necessary write permissions to the packages scope, allowing a workflow to authenticate and publish packages to GitHub Packages associated with the repository or organization.

E. to create issues in the repo.

The GITHUB_TOKEN automatically grants write permission to the issues scope, allowing a workflow to use the GitHub API to create, update, or close issues in the repository where the workflow is running.

Question: 17 CertyIQ

As a developer, which of the following snippets will enable you to run the commands npm ci and npm run build as part of a workflow?

A. - run: |
npm ci
npm run build

B. - shell:
npm ci
npm run build

C. - run: |
npm ci
npm run build
shell: nodejs

D. - run:
npm ci
npm run build

E.
- shell: |
npm ci
npm run build

Answer: C

Explanation:

The correct answer is **C. - run: |npm ci\nnpm run build\nshell: nodejs**. This is because it correctly specifies the steps to execute the npm commands within a GitHub Actions workflow.

Let's break down why the other options are incorrect and why option C is the most suitable:

Option A. - run: |npm cinpm run build: This option is incorrect because it attempts to execute npm ci and npm run build on the same line within the shell, without separating them as distinct commands. The | allows for multi-line strings in YAML, but the commands need to be on separate lines to be treated as separate commands.

Option B. - shell:npm cinpm run build: This option is incorrect as it uses shell: which isn't meant to directly execute commands. shell: defines which shell environment (e.g., bash, powershell) to use for the subsequent run steps within the workflow. It doesn't execute commands on its own.
Option D. - run:npm cinpm run build: This is incorrect because it attempts to run the commands on a single line without any separation, which would lead to a syntax error.
Option E. - shell: |npm cinpm run build: This is incorrect for similar reasons as option B. The shell keyword is used to specify which shell environment should be used for subsequent run steps, not to execute commands directly. The | introduces a multi-line string intended for shell configuration, not command execution.

Why Option C is Correct:

Option C uses the run: keyword, which is the correct way to execute shell commands within a GitHub Actions workflow step. The | character allows for a multi-line string in YAML, and the two commands npm ci and npm run build are placed on separate lines. This ensures that each command is executed independently. The shell: nodejs specifies that the script be executed within the nodejs enviroment so that npm can be used.

run:: This keyword instructs GitHub Actions to execute the specified commands within a shell environment.
|: This symbol allows for multi-line strings in YAML. It's essential for readability and for executing multiple commands in a single step.
npm ci: This command performs a clean install of dependencies based on package-lock.json or npm-shrinkwrap.json. This is generally preferred for CI/CD environments as it ensures consistent dependency versions and faster installation times.
npm run build: This command executes the build script defined in the package.json file. This typically compiles, bundles, and optimizes the application for deployment.
shell: nodejs: The shell is used to specify the runtime enviorment and that the script is to be run within that runtime enviorment.

Example Usage in a Workflow:

```
jobs:  build:    runs-on: ubuntu-latest    steps:      - name: Checkout code        uses: actions/checkout@v3      - name: Set u
```

Key Concepts:

GitHub Actions: A CI/CD platform directly integrated into GitHub repositories, allowing automated workflows based on events like code commits, pull requests, and scheduled tasks.
Workflow: A configurable automated process that will run one or more jobs.
Jobs: A set of steps that run on the same runner.
Steps: Individual tasks that can run commands, set up tools, or run actions.
run keyword: Used to execute shell commands.
shell keyword: Used to define the shell environment.
YAML: The data serialization language used to define GitHub Actions workflows.
npm: Node Package Manager, used to manage dependencies and run scripts in Node.js projects.
npm ci vs npm install: npm ci is designed for CI/CD, providing faster, cleaner, and more reliable installations based on the lockfile. npm install modifies the package-lock.json, which is undesired for a CI enviroment.

Supporting Links:

GitHub Actions Documentation: https://docs.github.com/en/actions
Understanding GitHub Actions: https://www.digitalocean.com/community/tutorials/an-introduction-to-github-actions
npm ci documentation: https://docs.npmjs.com/cli/v9/commands/npm-ci

Question: 18 CertyIQ

What is the smallest scope for an environment variable?

A. the workflow settings
B. the workflow env mapping
C. a job
D. a step

Answer: D

Explanation:

D. a step

In GitHub Actions, environment variable scope can be defined at different levels:

Workflow level → applies to all jobs/steps

Job level → applies to all steps in a job

Step level → applies only to a single step (smallest scope)

Question: 19 CertyIQ

Which workflow event is used to manually trigger a workflow run?

A. status
B. workflow_dispatch
C. workflow_run
D. create

Answer: B

Explanation:

B. workflow_dispatch

The workflow_dispatch event is used to manually trigger a GitHub Actions workflow from the GitHub UI or via API.

Question: 20 CertyIQ

Where should workflow files be stored to be triggered by events in a repository?

A. anywhere
B. Nowhere; they must be attached to an action in the GitHub user interface.
C. .github/workflows/

D. .workflows/

E. .github/actions/

Answer: D

Explanation:

D. .workflows/.

GitHub Actions workflow files must be stored in the .github/workflows/ directory in your repository to be automatically triggered by events (such as pushes, pull requests, or issue comments). This is the standard location where GitHub looks for workflow YAML files

Question: 21 CertyIQ

Your organization is managing secrets using GitHub encrypted secrets, including a secret named SuperSecret. As a developer, you need to create a version of that secret that contains a different value for use in a workflow that is scoped to a specific repository named MyRepo. How should you store the secret to access your specific version within your workflow?

A. Create MyRepo_SuperSecret in GitHub encrypted secrets to specify the scope to MyRepo.

B. Create a duplicate entry for SuperSecret in the encrypted secret store and specify MyRepo as the scope.

C. Create a file with the SuperSecret information in the .github/secrets folder in MyRepo.

D. Create and access SuperSecret from the secrets store in MyRepo.

Answer: A

Explanation:

Create MyRepo_SuperSecret in GitHub encrypted secrets to specify the scope to MyRepo," is the recommended practice. GitHub Actions prioritizes repository secrets over organization secrets when the repository secret name incorporates the repository name as a prefix. This means a secret named MyRepo_SuperSecret in the MyRepo repository will override the organization-level secret SuperSecret when the workflow runs within that repository. This allows for customization of shared secrets on a per-repository basis without modifying the organization-level secret itself.

Question: 23 CertyIQ

Which default GitHub environment variable indicates the owner and repository name?

A. GITHUB_REPOSITORY

B. GITHUB_WORKFLOW_REPO

C. ENV_REPOSITORY

D. REPOSITORY_NAME

Answer: A

Explanation:

The correct answer is **A. GITHUB_REPOSITORY**.

The GITHUB_REPOSITORY environment variable is a default environment variable automatically available in GitHub Actions workflows. It represents the repository slug, which is a combination of the owner (user or organization) and the repository name itself, formatted as owner/repository. For example, if your repository is owned by the user "myuser" and the repository is named "myrepo", the GITHUB_REPOSITORY variable would contain the value "myuser/myrepo". This variable is fundamental for identifying the context in which the workflow is running.

Question: 24 CertyIQ

You are a DevOps engineer in ABC Corp. You need to schedule your deployment workflow twice a week at 7:45 UTC every Wednesday and Saturday. What is the appropriate YAML structure?

A. on:
cron: '45 7 * * 3,6'

B. on:
cron:
- schedule: '45 7 * * 3,6'

C. on:
schedule:
- cron: '45 7 * * 3,6'

D. on:
schedule: '45 7 * * 3,6'

Answer: C

Explanation:

C. on:

schedule:

- cron: '45 7 * * 3,6'

To schedule workflows in GitHub Actions, you must use the schedule: keyword under on:, and each cron entry must be defined in a list using - cron.

Question: 25 CertyIQ

You need to create new workflows to deploy to an unfamiliar cloud provider. What is the fastest and safest way to begin?

A. Create a custom action to wrap the cloud provider's CLI.
B. Search GitHub Marketplace for verified actions published by the cloud provider.
C. Use the actions/jenkins-plugin action to utilize an existing Jenkins plugin for the cloud provider.
D. Search GitHub Marketplace for actions created by GitHub.
E. Download the CLI for the cloud provider and review the associated documentation.

Answer: B

Explanation:

B. Search GitHub Marketplace for verified actions published by the cloud provider.

When you need to deploy to a new or unfamiliar cloud provider, the fastest and safest way to get started is to use a verified action from the GitHub Marketplace — especially if it's published by the official cloud provider.

These actions:

Are maintained and supported by the provider.

Have been security-reviewed and verified by GitHub.

Allow you to quickly integrate without writing or maintaining custom scripts.

Question: 26 CertyIQ

Which scopes are available to define custom environment variables within a workflow file? (Choose three.)

A. the entire workflow, by using env at the top level of the workflow file
B. all jobs being run on a single Actions runner, by using runner.env at the top of the workflow file
C. the entire stage, by using env at the top of the defined build stage
D. within the run attribute of a job step
E. the contents of a job within a workflow, by using jobs.env
F. a specific step within a job, by using jobs..steps[*].env

Answer: AEF

Explanation:

A. the entire workflow, by using env at the top level of the workflow file

Correct. This defines environment variables available to all jobs and steps:

env:

GLOBAL_VAR: value

E. the contents of a job within a workflow, by using jobs.<job_id>.env

Correct. While the option says jobs.env (slightly imprecise), it clearly refers to job-level env, which is valid:

jobs:

build:

env:

JOB_VAR: value

F. a specific step within a job, by using jobs.<job_id>.steps[*].env

Correct. This scopes variables to a single step:

steps:

- name: Step example

env:

STEP_VAR: value

Question: 28 CertyIQ

Which of the following scenarios requires a developer to explicitly use the GITHUB_TOKEN or github.token secret within a workflow? (Choose two.)

A. passing the GITHUB_TOKEN secret to an action that requires a token as an input
B. making an authenticated GitHub API request
C. checking out source code with the actions/checkout@v3 action
D. assigning non-default permissions to the GITHUB_TOKEN

Answer: AB

Explanation:

Here's a detailed justification for the correct answers:

A. Passing the GITHUB_TOKEN secret to an action that requires a token as an input:

This is a common scenario. Many GitHub Actions require a token to interact with the GitHub API on your behalf. These actions often have an input parameter specifically for a token. You explicitly pass the GITHUB_TOKEN secret to this input parameter to authorize the action to perform tasks like creating issues, commenting on pull requests, or updating deployments. Without passing a token, the action would lack the necessary permissions to interact with GitHub resources.

B. Making an authenticated GitHub API request:

When you want your workflow to directly interact with the GitHub API (e.g., using curl or a dedicated GitHub API client), you must provide authentication. The GITHUB_TOKEN is the default way to authenticate these requests within a workflow. You'll typically include the token in the Authorization header of your API request. This tells the GitHub API that the workflow is authorized to perform the requested action on behalf of the repository. Failing to authenticate API requests results in errors, typically HTTP 401 Unauthorized or 403 Forbidden.

Why other options are incorrect:

C. Checking out source code with the actions/checkout@v3 action:

The actions/checkout@v3 action implicitly uses the GITHUB_TOKEN to clone the repository by default. You don't explicitly need to pass it as an input. The action automatically handles authentication for basic checkout operations within the same repository where the workflow is running. However, if you need to checkout another repository, or you need to perform more complex operations that require increased permissions (like pushing commits), then you might need to use a PAT (Personal Access Token) and explicitly pass it to the actions/checkout@v3 action.

D. Assigning non-default permissions to the GITHUB_TOKEN:

While you can assign custom permissions to the GITHUB_TOKEN using the permissions key in your workflow file, this doesn't inherently mean you need to explicitly pass or specify the token itself. You are configuring what the default token can do, not passing the token itself to another action or API call. The assigned permissions influence what the GITHUB_TOKEN (implicitly used by some actions) can accomplish.

Supporting Resources:

1. **About the GITHUB_TOKEN:** https://docs.github.com/en/actions/security-guides/automatic-token-authentication
2. **GitHub Actions Permissions:** https://docs.github.com/en/actions/using-jobs/assigning-permissions-to-jobs
3. **actions/checkout:** https://github.com/actions/checkout

Question: 29 CertyIQ

As a developer, you have a 10-MB data set that is required in a specific workflow.
Which three steps should you perform so the dataset is stored encrypted and can be decrypted during the workflow? Each correct answer presents part of the solution.
NOTE: Each correct answer is worth one point.

A. Store the encryption keys in a GitHub encrypted secret.
B. Encrypt the dataset.
C. Leverage the actions/download-secret action in the workflow.
D. Store the dataset in a GitHub encrypted secret.
E. Create a GitHub encrypted secret with the Large object option selected and upload the dataset.
F. Compress the dataset.
G. Commit the encrypted dataset to the same repository as the workflow.

Answer: ABG

Explanation:

Technical Justification for Correct Answer: ABG

The correct sequence to store a 10-MB dataset encrypted and decrypt it during a GitHub Actions workflow involves three key steps: **A (Store the encryption keys in a GitHub encrypted secret)**, **B (Encrypt the dataset)**, and **G (Commit the encrypted dataset to the same repository as the workflow)**. Below is a detailed explanation of why these steps are optimal and why other options are less suitable.

Correct Steps Explained

A. Store the encryption keys in a GitHub encrypted secret:

Why Correct: Storing encryption keys as GitHub encrypted secrets ensures they are not exposed in plaintext within the workflow files or logs, maintaining security.
Technical Justification: GitHub encrypts secrets at rest and in transit, and they are only accessible to actions within the workflow, reducing the attack surface.

B. Encrypt the dataset:

Why Correct: Encrypting the dataset protects it from unauthorized access both at rest (in the repository) and

potentially in transit if downloaded elsewhere.
Technical Justification: Encryption (e.g., using AES) ensures confidentiality. For GitHub Actions, tools like openssl can be used for encryption/decryption.

G. Commit the encrypted dataset to the same repository as the workflow:

Why Correct: Committing the encrypted dataset to the repository alongside the workflow ensures availability for the workflow without needing external storage solutions.
Technical Justification: Given the dataset's size (10 MB) is manageable for most repositories' storage limits, and it's encrypted, the risk is mitigated. GitHub Actions can then access it directly.
Why Other Options Are Less Suitable

C. Leverage the actions/download-secret action in the workflow:

Less Suitable: This action is for downloading secrets, not datasets. While you'd use a similar concept for keys (hence why A is correct for key storage), the dataset itself shouldn't be stored as a secret due to size and usage limitations.

D. Store the dataset in a GitHub encrypted secret:

Less Suitable: Secrets are intended for small, sensitive data (like keys or tokens), not large datasets. GitHub has size limits for secrets, making this impractical for a 10-MB dataset.

E. Create a GitHub encrypted secret with the Large object option selected and upload the dataset:

Less Suitable: While this mentions a "Large object option," GitHub Secrets are not designed for large data storage. This approach would be against best practices for secret usage and might hit operational limits.

F. Compress the dataset:

Less Suitable for the Question's Context: While compression is generally a good practice for reducing storage and transfer sizes, the question focuses on encryption for security, not size optimization. Compression could be an additional step but doesn't address the encryption requirement directly.
References

GitHub Documentation: Encrypting secrets in your repository
GitHub Documentation: Actions - Downloading a secret

Question: 30 CertyIQ

For which type of GitHub Actions workflows should you use a self-hosted runner?

A. workflows that require access to private internal network resources
B. workflows that have no specific environment configurations and must be triggered on a GitHub-hosted infrastructure
C. short-lived workflows that do NOT require custom dependencies
D. workflows that require the latest operating system and preinstalled software versions

Answer: A

Explanation:

Justification for Correct Answer: A

Why A is the best option:A self-hosted runner is most suitable for **workflows that require access to private internal network resources**. This is because self-hosted runners can be installed on your own servers or virtual machines within your organization's network, allowing GitHub Actions workflows to access resources that are not exposed to the internet, such as private databases, internal APIs, or restricted file shares. GitHub-hosted runners, on the other hand, run in isolated environments provided by GitHub and cannot directly access your private internal network.

Why other options are less suitable:

B. Workflows with no specific environment configurations and must be triggered on a GitHub-hosted infrastructure: This scenario is more suited for **GitHub-hosted runners** since there are no specific environment requirements, and the need for a hosted infrastructure is explicitly mentioned, making a self-hosted runner unnecessary.

C. Short-lived workflows that do NOT require custom dependencies: **GitHub-hosted runners** are ideal for short-lived workflows without custom dependency needs. They provide a quick, ephemeral environment with pre-configured dependencies for common workflows, efficient for transient tasks.

D. Workflows that require the latest operating system and preinstalled software versions: While it might seem beneficial to use a self-hosted runner for control over the environment, **GitHub-hosted runners** frequently update their operating systems and software packages to provide the latest versions. Unless the very latest versions (not yet available on GitHub-hosted runners) are required, or specific, uncommon software is needed, GitHub-hosted runners would be more convenient and cost-effective for this scenario.

Key Points Summary (Bullet Points for Quick Reference)

Self-Hosted Runner (Correct Choice - A):

Necessary for access to **private internal network resources**.
Provides control over the runtime environment.
GitHub-Hosted Runner (More Suitable for B, C, and potentially D):

B: No specific env configs + need for hosted infra.
C: Short-lived + no custom deps.
D: Latest OS/software (if not excessively bleeding-edge or uncommon).

References

GitHub Documentation: Self-hosted runners
GitHub Documentation: Choosing between self-hosted and GitHub-hosted runners

Question: 31 CertyIQ

Which of the following scenarios would require the use of self-hosted runners instead of GitHub-hosted runners?

A. running more than the three concurrent workflows supported by GitHub-hosted runners
B. performing builds on macOS
C. exceeding 50,000 monthly minutes of build time
D. using specialized hardware configurations required for workflows
E. using Docker containers as part of the workflow

Answer: D

Explanation:

Technical Justification for Choosing Option D

The correct answer is **D. using specialized hardware configurations required for workflows**. Here's why:

Option D: Self-hosted runners are necessary when workflows demand specialized hardware configurations (e.g., specific GPUs, high-end CPUs, or custom network setups) not provided by GitHub-hosted runners. GitHub-hosted runners offer a standardized environment, lacking the flexibility to accommodate custom hardware needs. Thus, self-hosted runners are the only viable option for such scenarios.

Why Other Options are Less Suitable for Self-Hosted Runners:

A. running more than the three concurrent workflows supported by GitHub-hosted runners: While a potential limitation, exceeding the concurrent workflow limit can often be managed by optimizing workflow efficiency, using GitHub's auto-scaling for hosted runners (for large plans), or purchasing additional concurrency minutes, making self-hosted runners not the only solution.
B. performing builds on macOS: GitHub-hosted runners **do** support macOS as one of their environments (along with Windows and Ubuntu), so there's no necessity for self-hosted runners solely for macOS builds.
C. exceeding 50,000 monthly minutes of build time: This is a billing and plan limitation. Upgrading to a more comprehensive GitHub plan or optimizing build times can mitigate this, rather than necessitating a switch to self-hosted runners.
E. using Docker containers as part of the workflow: Both GitHub-hosted and self-hosted runners support the use of Docker containers within workflows. GitHub-hosted runners can run Docker-in-Docker (though with some limitations), and self-hosted runners offer more control but are not required solely for Docker usage.

Conclusion: The primary scenario necessitating self-hosted runners over GitHub-hosted ones is when the workflow requires **specialized hardware configurations** not available in the standardized environment of GitHub-hosted runners.

References

1. **GitHub Documentation - About Self-hosted Runners**: https://docs.github.com/en/actions/hosting-your-own-runners/about-self-hosted-runners
2. **GitHub Documentation - About GitHub-hosted Runners**: https://docs.github.com/en/actions/hosting-your-own-runners/about-github-hosted-runners

Question: 32 CertyIQ

You are developing a GitHub Actions workflow that will build a .NET app.
You need to provide the workflow with access to a sensitive enterprise license. The solution must ensure that the license is NOT exposed publicly or hardcoded.
What should you do?

A. Use a GitHub-hosted Windows runner and expose the license via a public URL downloaded by the workflow at runtime.

B. Embed the license into an encrypted ZIP file stored in a repository and decrypt the file at runtime by using a hardcoded passphrase.

C. Use GitHub Enterprise Managed User (OIDC) in a GitHub-hosted or self-hosted Windows runner to fetch the license from a third-party vault or secrets manager at runtime.

D. Use a GitHub-hosted Ubuntu runner and store the license in a Base64-encoded repository.

Answer: C

Explanation:

Technical Justification for Correct Option (C)

Why C is the best choice: The correct option, C, is the most suitable because it utilizes **GitHub Enterprise Managed User (OIDC)**, which securely fetches the license from a **third-party vault or secrets manager** at runtime. This approach ensures the sensitive enterprise license is:

Not exposed publicly
Not hardcoded in the workflow or repository
Retrieved securely using OIDC, which provides a secure, temporary token-based authentication without storing credentials directly in the workflow.
Compatible with both GitHub-hosted and self-hosted Windows runners, offering flexibility in the deployment environment.

Why other options are less suitable:

A. Use a GitHub-hosted Windows runner and expose the license via a public URL:

Public Exposure: The license would be exposed publicly via the URL, which is a significant security risk.
No Security: Downloading from a public URL does not protect the license from unauthorized access.

B. Embed the license into an encrypted ZIP file with a hardcoded passphrase:

Hardcoded Passphrase: Despite encryption, hardcoding the passphrase in the workflow or repository compromises security, as it can be discovered through repository history or access.
Limited Security: While encryption helps, the hardcoded passphrase undermines the overall security posture.

D. Use a GitHub-hosted Ubuntu runner and store the license in a Base64-encoded repository:

Base64 is Not Encryption: Base64 encoding is for data format conversion, not security. It does not protect the license from being read or used if accessed.
Public Exposure Risk: If the repository is public or accessed inappropriately, the license can be easily decoded and exposed.

References:

For deeper understanding of the recommended approach and security best practices with GitHub Actions:

1. **GitHub Docs - Storing and using sensitive values**: https://docs.github.com/en/actions/security-guides/storing-and-using-sensitive-values
2. **GitHub Docs - About hosted runners**: https://docs.github.com/en/actions/using-workflow-runners/about-hosted-runners#about-github-hosted-runners (Focus on security and OIDC for managed users)

Question: 33 CertyIQ

Which YAML snippet triggers a GitHub Actions workflow for both push and pull request events?

A.

```
on:
  push:
    branches:
      - dev
  pull_request:
    branches:
      - main
```

B.

```
on:
  push:
    branches:
      - main
```

C.

```
on:
  push:
    branches:
      - main
  push:
    branches:
      - dev
```

D.

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

Answer: D

Explanation:

```
on:
  push:
    branches:
      - main
  pull_request:
    branches:
      - main
```

Question: 34 CertyIQ

You need to stop a GitHub Actions workflow from running. The solution must retain the configuration and execution history of the workflow.
What should you do?

A. Delete the workflow YAML file from the repository.
B. Select Actions and disable the workflow.
C. Rename the workflow YAML file.
D. Archive the workflow in a new branch.

Answer: B

Explanation:

Technical Justification for Correct Option (B)

Why B is the Best Option: Selecting **Actions** and **disabling the workflow** (Option B) is the most appropriate solution for stopping a GitHub Actions workflow while retaining both its configuration and execution history. This approach directly targets the workflow's activation state without altering or removing the underlying configuration (YAML file) or execution records. Disabling ensures that:

The workflow YAML file remains in the repository, preserving the configuration.
The execution history is untouched, allowing for auditing, debugging, or future reference.
The workflow can be easily re-enabled if needed, maintaining its current state.

Why Other Options are Less Suitable:

A. Delete the workflow YAML file from the repository:

Configuration Loss: Deletes the workflow configuration, contrary to the requirement of retaining it.
Irreversible (without version control recovery): Unless retrieved from version control history, the configuration is permanently lost.

C. Rename the workflow YAML file:

Configuration Integrity: While the file remains, renaming it would prevent GitHub from recognizing it as a valid workflow file (unless the name change follows specific naming conventions, which is not guaranteed to stop execution immediately).
Unreliable Stop: GitHub might still attempt to run the workflow if the rename doesn't perfectly align with expected naming conventions or if there's a delay in recognizing the change.

D. Archive the workflow in a new branch:

Operational Complexity: Involves unnecessary repository management steps (creating a new branch, archiving) that don't directly address stopping the workflow.
Configuration and History Accessibility: While the configuration and history are retained, accessing or managing them becomes less straightforward compared to simply disabling the workflow.

Correct Answer Justification Summary: Option B, **Select Actions and disable the workflow**, is the most direct, non-destructive, and efficient method to stop a GitHub Actions workflow while fulfilling the retention requirements for both configuration and execution history.

References:

GitHub Actions - Disabling a workflow
GitHub Actions - Workflow files

Question: 35 CertyIQ

You need to publish a release that includes assets in GitHub Actions. The solution must meet GitHub support requirements.
What should you do?

A. Run actions/upload-artifact followed by actions/create-release, and then actions/upload-release-asset.
B. Run gh release create in a script step to create a release and attach the assets after tagging a commit.
C. Use a GitHub Marketplace action that creates a release and uploads the assets in a single step.
D. Run git push --tags to automatically trigger GitHub to create a release that includes all the project files.

Answer: B

Explanation:

Technical Justification for Correct Answer: B

The correct approach to publish a release that includes assets in GitHub Actions, meeting GitHub support requirements, is **option B**. Here's why:

Why B is the best choice:

Native Integration and Simplicity: Using gh release create in a script step (as part of a GitHub Actions workflow) to create a release and then attaching assets leverages GitHub's CLI tool, ensuring a straightforward and supported method for release management.
Control and Flexibility: This approach allows for precise control over when the release is created (after tagging a commit) and which assets are attached, aligning with common CI/CD pipeline requirements.
GitHub Support Compliance: Since gh is an official GitHub CLI, using it complies with GitHub's support requirements, minimizing potential issues with future updates or support needs.

Why other options are less suitable:

A. actions/upload-artifact, actions/create-release, then actions/upload-release-asset:

Additional Complexity: Involves multiple steps with separate actions, increasing the workflow's complexity unnecessarily.
Artifact Upload Limitation: Upload artifacts are primarily for workflow artifacts, not necessarily optimized for release assets in this context.

C. GitHub Marketplace action for a single step:

Dependency on Third-Party: Introduces a dependency on a potentially less stable or maintained third-party action, which might not always align with the latest GitHub features or support guidelines.
Less Transparent Control: Single-step solutions can obscure the process, making debugging and customization harder.

D. git push --tags to trigger automatic release:

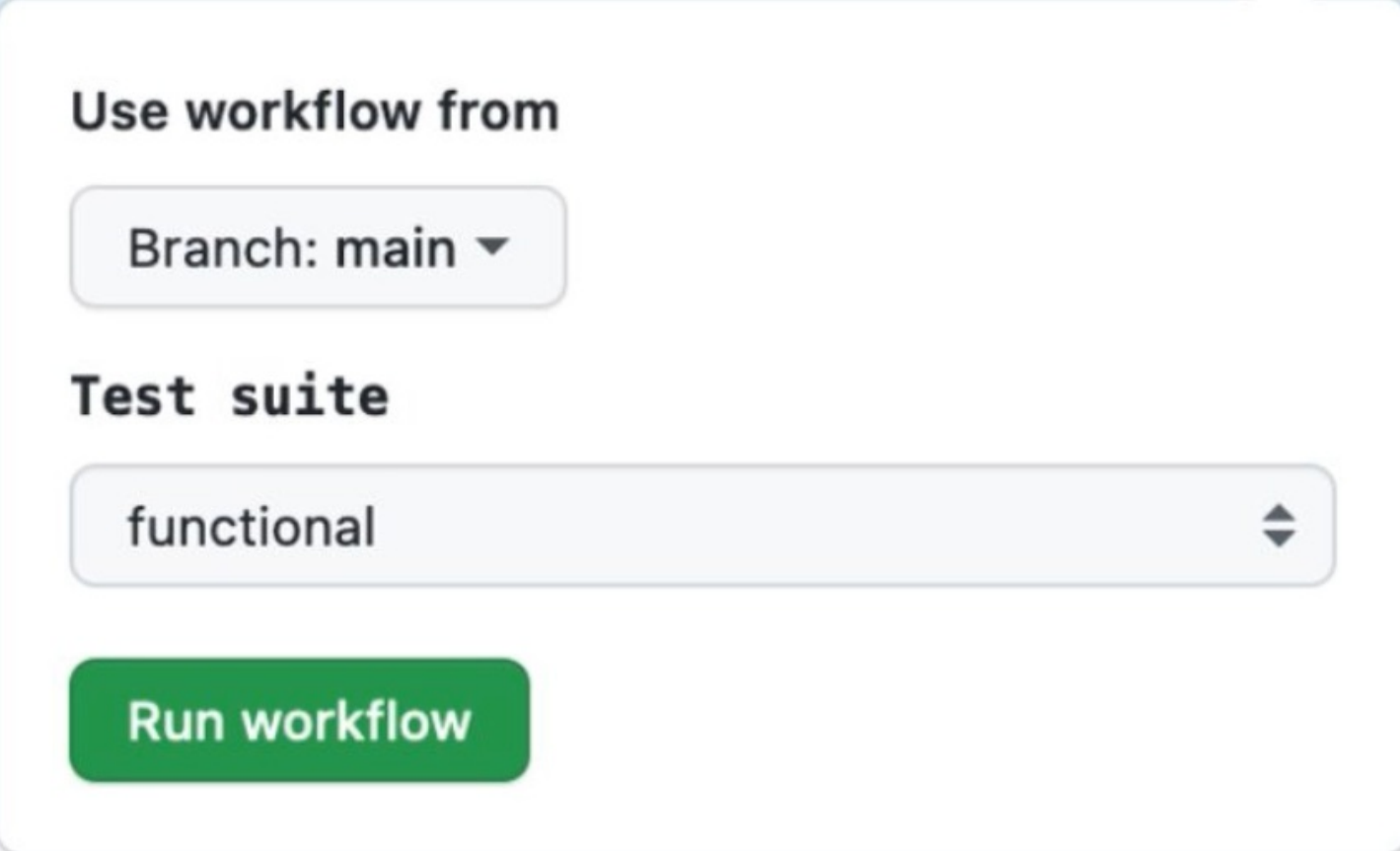
Lack of Control Over Release Content: Automatically includes all project files, which is often not desirable. Releases typically require curated assets.
Dependence on GitHub Actions Workflow for Asset Control is Lost: This method doesn't integrate well with the precise asset management typically handled within a GitHub Actions workflow.

References:

GitHub CLI Documentation - Releases
GitHub Actions - Uploading artifacts

Question: 36 CertyIQ

As a developer, you want to run a workflow from the Actions tab in GitHub. Which YAML snippet should you use to match the interface in this image?



A.

```
on:
  workflow_run:
    inputs:
      test_suite:
        description: Test suite
        type: string
        options:
          - functional
          - regression
```

B.

```
on:
  workflow_dispatch:
    inputs:
      test_suite:
        description: Test suite
        type: choice
        options:
          - functional
          - regression
```

C.

```
on:
  workflow_run:
    inputs:
      test_suite:
        description: Test suite
        type: choice
        options:
          - functional
          - regression
```

```
on:
  workflow_dispatch:
    inputs:
      test_suite:
        description: Test suite
        type: choice
        value: functional
        options:
          - regression
```

D.

Answer: B

Explanation:

```
on:
  workflow_dispatch:
    inputs:
      test_suite:
        description: Test suite
        type: choice
        options:
          - functional
          - regression
```

Question: 37 CertyIQ

Which statement accurately describes using labels to route GitHub Actions workflows to runners?

A. A user can assign multiple labels to a self-hosted runner and use the labels in a runs-on to target only runners that match all the specified labels.

B. Labels apply only to GitHub-hosted runners and are ignored by self-hosted runners.

C. GitHub Actions automatically assigns the runner's IP address as a label, and the label must be used in a runs-on.

D. To target a specific runner, the runs-on must match the runner's hostname rather than its labels.

Answer: A

Explanation:

Option A is correct because GitHub Actions supports multi-label matching for self-hosted runners. When configuring a self-hosted runner, administrators can assign multiple custom labels (e.g., linux, gpu, azure). In the workflow YAML, specifying runs-on: [linux, gpu] targets only runners possessing **all** those labels simultaneously. This allows precise routing based on capabilities, architecture, or environment without needing unique runner groups for every permutation.

Option B is incorrect because labels are a fundamental routing mechanism for **self-hosted runners**, not GitHub-hosted runners. GitHub-hosted runners are selected via standardized virtual environment labels (e.g., ubuntu-latest, windows-2022), whereas self-hosted runners rely heavily on custom and default labels (like self-hosted, X64, Linux) for selection.

Option C is incorrect because GitHub Actions does not automatically assign the runner's IP address as a label, nor does it require IP-based targeting in runs-on. Labels are user-defined metadata strings (or default system labels like OS/Arch); network addressing is handled internally by the runner-controller communication and is not exposed as a routing label.

Option D is incorrect because runs-on matches against **labels**, not the runner's hostname. While the hostname is visible in the runner's UI details and logs, the routing engine exclusively evaluates the label set assigned to the runner against the runs-on array in the workflow definition.

References

Using labels with self-hosted runners
Workflow syntax for GitHub Actions: jobs..runs-on

Question: 38 CertyIQ

For which type of GitHub Actions workflows should you use a self-hosted runner?

A. workflows on a wide range of open-source software preinstalled in a GitHub-hosted environment

B. workflows in environments that have specific hardware or operating system requirements that are unavailable on a GitHub-hosted runner

C. workflows that require high availability and automatic scaling

D. workflows that have standard configurations and frequent GitHub updates

Answer: B

Explanation:

Justification for Correct Answer (B)

Self-hosted runners are required when workflows demand execution environments that GitHub-hosted runners cannot provide. This includes specific hardware architectures (e.g., ARM, GPU, specialized accelerators), proprietary operating system images, strict network isolation (e.g., access to internal private networks/VPNs behind firewalls), or persistent state/caching on local storage that survives job completion. GitHub-hosted runners offer standardized, ephemeral, and immutable images (Ubuntu, Windows, macOS) with

fixed hardware specifications (2-core/4-core CPU, 7GB/14GB RAM), making them incompatible with these specialized infrastructure requirements.

Why Other Options Are Less Suitable

A. Workflows on a wide range of open-source software preinstalled in a GitHub-hosted environment: This describes the primary value proposition of **GitHub-hosted runners**. They come preloaded with extensive toolchains (languages, runtimes, CLI tools) and are the default choice for standard OSS builds. Using self-hosted runners here introduces unnecessary operational overhead (maintenance, patching, scaling) without benefit.
C. Workflows that require high availability and automatic scaling: GitHub-hosted runners provide **automatic, elastic scaling** and high availability managed entirely by GitHub (SLA-backed). Self-hosted runners require manual implementation of autoscaling solutions (e.g., Actions Runner Controller on Kubernetes, VM scale sets) and the operator assumes full responsibility for uptime, capacity planning, and infrastructure health.
D. Workflows that have standard configurations and frequent GitHub updates: Standard configurations are the ideal use case for **GitHub-hosted runners**, which receive automatic OS patches, tool updates, and security fixes from GitHub. Self-hosted runners require the administrator to manually apply OS updates, runner application updates, and toolchain upgrades, creating maintenance latency and security drift risk.

References

1. GitHub Docs: About self-hosted runners - Differences between GitHub-hosted and self-hosted runners
2. GitHub Docs: Using GitHub-hosted runners - Supported runners and hardware resources

Thank you

Thank you for being so interested in the premium exam material.
I'm glad to hear that you found it informative and helpful.

But Wait

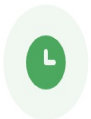
I wanted to let you know that there is more content available in the full version.
The full paper contains additional sections and information that you may find helpful, and I encourage you to download it to get a more comprehensive and detailed view of all the subject matter.

Download Full Version Now



Future is Secured

100% Pass Guarantee



24/7 Customer Support

Mail us - certyiqofficial@gmail.com



Free Updates

Lifetime Free Updates!

Total: **126 Questions**

Link: https://certyiq.com/papers/microsoft/gh-200